AI-Powered Social Listening & Reputation Monitoring Platform

Complete Technical Planning · High Level Design · API Reference · DB Schema

Production-Grade · Microservices · Event-Driven · Cloud-Native · AI-Powered

FastAPI | React + TypeScript | Apache Kafka | Redis | PostgreSQL

gRPC | Docker | AWS ECS/EKS | Prometheus

Table of Contents

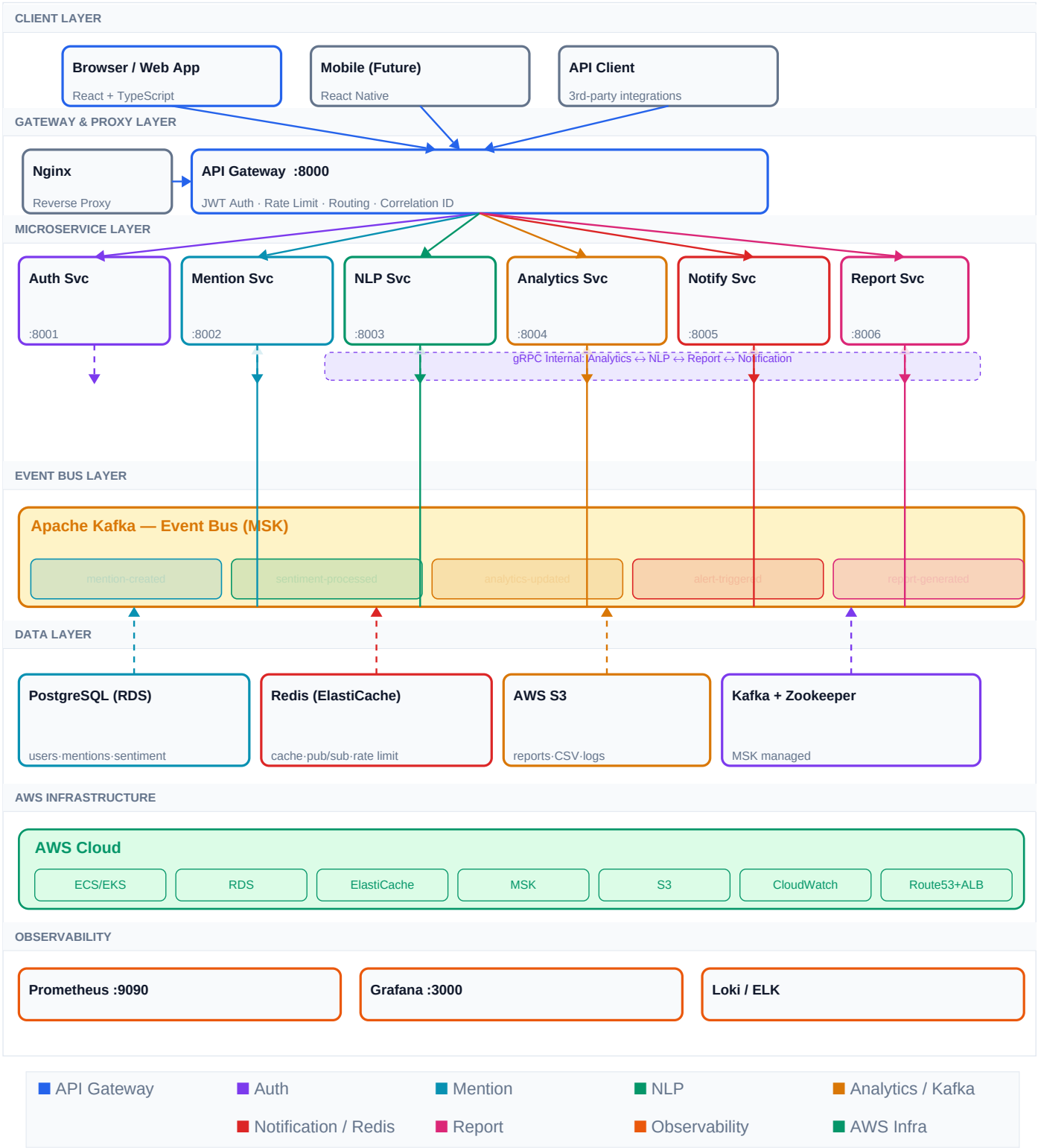
1. Project Overview & Vision

This platform continuously monitors public internet sources, detects brand/product/company mentions, performs AI-based sentiment analysis, triggers real-time alerts, and visualises analytics via live dashboards. It is designed to production-grade standards, demonstrating distributed systems, event-driven architecture, AI integration, WebSocket communication, cloud deployment, and full-stack engineering.

Feature	Description	Technology
JWT Auth + RBAC	Role-based access (Admin/Analyst/Viewer)	FastAPI · PyJWT · bcrypt
Mention Tracking	Track brands, products, people across sources	Scrapy · PRAW · Playwright
Web Scraping Pipeline	Reddit, NewsAPI, RSS, blogs with deduplication	Celery · Redis bloom filter
AI Sentiment Analysis	Positive / Negative / Neutral + confidence score	HuggingFace · spaCy · GPT-4o
Spike Detection	Z-score anomaly detection on mention volume	Pandas · NumPy
Live Dashboard	Real-time charts updated via WebSocket	React · Recharts · WebSocket
Notifications	Email + WebSocket alerts on rule triggers	SendGrid · Redis Pub/Sub
PDF / CSV Reports	Background generation with S3 storage	WeasyPrint · Pandas · boto3
Competitor Analysis	Side-by-side brand comparison scoring	Analytics Service · gRPC

2. High Level Architecture Diagram

Client requests flow through Nginx into the API Gateway. The gateway handles JWT validation, rate limiting, and routing to 7 independent microservices. Services communicate asynchronously via Apache Kafka and synchronously via gRPC. Redis handles caching, sessions, rate limiting and WebSocket pub/sub bridging. All services run as Docker containers on AWS ECS/EKS.



3. Technology Stack

Layer	Technology	Justification
Frontend	React.js + TypeScript + Tailwind CSS	Type-safe UI, component model, utility-first CSS
Backend	FastAPI (Python 3.11)	Async, auto OpenAPI docs, Pydantic validation
Message Queue	Apache Kafka	Durable, partitioned, high-throughput event log
Cache/PubSub	Redis 7	Sub-ms latency, pub/sub, atomic counters
Database	PostgreSQL 15	ACID, JSONB support, full-text search, partitioning
Internal RPC	gRPC + Protocol Buffers	Type-safe, efficient binary protocol, streaming
Containers	Docker + Docker Compose	Reproducible environments, service isolation
Deployment	AWS ECS / EKS	Managed containers, auto-scaling, ALB integration
Reverse Proxy	Nginx	TLS termination, static files, upstream balancing
Monitoring	Prometheus + Grafana	Pull-based metrics, rich dashboards, alerting
Logging	Loki / ELK Stack	Centralised log aggregation, correlation tracing
Task Queue	Celery + Redis	Async background jobs, scheduled scraping crons
CI/CD	GitHub Actions	Test → build → push ECR → deploy ECS on merge
Object Storage	AWS S3	Durable report/log storage, pre-signed URLs

4. Microservices Breakdown

4.1 API Gateway — Port :8000

Stack: FastAPI · Python 3.11 · Redis · httpx · Nginx

- Single entry point for all client HTTP + WebSocket traffic
- JWT validation on every protected route; attaches user context (user_id, role, org_id) to Request.state
- Routes requests to downstream services via httpx.AsyncClient with timeout + retry
- Redis sliding-window rate limiting: 100 req/min per user; returns 429 + Retry-After header
- Generates X-Correlation-ID for distributed tracing; logged on every request/response
- CORS handling, HSTS, X-Frame-Options, X-Content-Type-Options security headers
- GET /health: polls all downstream /health endpoints; used by ALB target group health check

```
ENV VARS: JWT_SECRET | REDIS_URL | SERVICE_URLS | RATE_LIMIT_MAX=100 | LOG_LEVEL=INFO
```

4.2 Auth Service — Port :8001

Stack: FastAPI · SQLAlchemy (async) · Alembic · bcrypt · PyJWT · PostgreSQL

- User registration with email uniqueness check; sends verification email via SendGrid
- Login: bcrypt.checkpw (constant-time), issues access token (15 min) + refresh token (7 days)
- Refresh tokens stored as Redis HASH keys; rotated on every /refresh call
- Token blacklisting on logout: Redis SET key matching JWT TTL for instant invalidation
- RBAC roles: Admin (full org management), Analyst (create/manage keywords), Viewer (read-only)
- OAuth2 social login via Authlib (Google, GitHub); auto-creates org on first login
- TOTP-based 2FA: generate QR code, verify TOTP, store encrypted secret in DB

```
ENV VARS: DB_URL | JWT_SECRET | JWT_EXPIRE_MINUTES=15 | REFRESH_EXPIRE_DAYS=7 | REDIS_URL | SMTP_HOST |
OAUTH_GOOGLE_CLIENT_ID | OAUTH_GOOGLE_CLIENT_SECRET
```

4.3 Mention Collection Service — Port :8002

Stack: FastAPI · Scrapy · Celery Beat · Playwright · PRAW · Redis · Kafka

- Manages keyword/brand/product tracking configs; enforces org plan max_keywords limit
- Celery Beat scheduler: configurable cron per keyword; Flower dashboard at :5555
- Reddit scraper: PRAW OAuth2 client credentials; fetches posts + comments matching keyword
- NewsAPI scraper: /everything endpoint; parses title, description, content, publishedAt
- RSS/Blog scraper: feedparser for 50+ feeds; Playwright headless Chromium for JS-rendered pages
- URL-hash deduplication: SHA256(source_url) as Redis SET key with 24h TTL; UNIQUE DB constraint
- Distributed scrape lock via Redis SETNX per keyword_id; prevents parallel duplicate jobs
- Publishes mention-created Kafka event after DB insert; uses org_id as partition key

```
ENV VARS: DB_URL | REDIS_URL | KAFKA_BROKERS | REDDIT_CLIENT_ID | REDDIT_CLIENT_SECRET | NEWSAPI_KEY |
SCRAPE_INTERVAL_MINUTES=30 | CELERY_BROKER_URL
```

4.4 NLP / AI Service — Port :8003

Stack: FastAPI · HuggingFace Transformers · spaCy · OpenAI API · Kafka · Celery

- Kafka consumer on mention-created; consumer group: nlp-service; manual commit after DB write
- Sentiment: cardiffnlp/twitter-roberta-base-sentiment-latest; batches of 32; GPU-aware
- Returns positive/negative/neutral with per-class confidence scores (0.0–1.0)
- Named Entity Recognition via spaCy en_core_web_sm; extracts ORG, PERSON, GPE, PRODUCT
- Keyword extraction: TF-IDF + KeyBERT (semantic); top 5 deduplicated keywords returned
- Language detection via langdetect; routes non-English text to multilingual model (XLM-R)
- GPT-4o-mini summarisation for content > 500 chars; cached in Redis with 1h TTL
- Publishes sentiment-processed Kafka event after persisting results to sentiment_results table

```
ENV VARS: KAFKA_BROKERS | DB_URL | OPENAI_API_KEY | HF_MODEL_NAME | GPU_ENABLED=false | MAX_TEXT_LENGTH=512 | SUMMARY_CACHE_TTL=3600
```

4.5 Analytics Service — Port :8004

Stack: FastAPI · Pandas · NumPy · Kafka · Redis · PostgreSQL · gRPC server

- Kafka consumer on sentiment-processed; updates rolling counters per (org_id, keyword, window)
- Hourly/daily/weekly mention aggregation with Pandas groupby; persists to time-series table
- Sentiment trend: % positive/negative per rolling window (1h, 24h, 7d configurable)
- Spike detection: Z-score — fires alert if count > mean + 2×std over 7-day rolling baseline
- Keyword co-occurrence matrix and frequency ranking; competitor normalised scoring
- 7 REST endpoints: /overview /trends /sentiment /keywords/top /spikes /competitors /sources
- Redis cache-aside with 5-min TTL; invalidated on analytics-updated event
- Publishes alert-triggered when spike_detected=true; analytics-updated every 5 minutes

```
ENV VARS: KAFKA_BROKERS | DB_URL | REDIS_URL | SPIKE_THRESHOLD=2.0 | TREND_WINDOW_HOURS=24 | CACHE_TTL=300
```

4.6 Notification Service — Port :8005

Stack: FastAPI · WebSockets · Redis Pub/Sub · SendGrid · Kafka

- WebSocket endpoints /ws/dashboard and /ws/alerts; JWT auth on HTTP upgrade handshake
- Redis Pub/Sub bridge: Kafka event → redis.publish(f'ws:{org_id}') → all connected WS clients
- Horizontal scale: any service instance serves any user via shared Redis channel (no sticky sessions)
- Alert evaluation engine: checks analytics-updated events against all enabled org rules
- Debounce: max 1 alert per rule per 10 minutes to prevent alert storms
- SendGrid transactional HTML email with Jinja2 template (spike summary, mention count, chart)
- Alert history API with cursor-based pagination (keyset on triggered_at)
- Kafka consumers: alert-triggered and report-generated topics

```
ENV VARS: KAFKA_BROKERS | DB_URL | REDIS_URL | SENDGRID_API_KEY | WS_HEARTBEAT_INTERVAL=30 | EMAIL_FROM_ADDRESS
```

4.7 Report Service — Port :8006

Stack: FastAPI · WeasyPrint · Pandas · Matplotlib · boto3 · S3 · Celery

- POST /reports returns 202 Accepted + report_id; Celery task handles actual generation
- PDF: Jinja2 HTML template rendered by WeasyPrint; Matplotlib charts embedded as base64 PNG
- PDF contents: cover page, KPI summary cards, mention trend chart, top keywords, sentiment table
- CSV: Pandas DataFrame JOIN mentions + sentiment_results; StreamingResponse for large exports
- boto3 multipart upload to S3 under org-scoped prefix (reports/{org_id}/{report_id}.pdf)
- Generates 24h pre-signed S3 URL; stores s3_key + expires_at in DB
- Recurring reports: Celery Beat CRON schedule (daily/weekly); emails download link on completion
- Publishes report-generated Kafka event on success (triggers email notification)

```
ENV VARS: DB_URL | REDIS_URL | KAFKA_BROKERS | AWS_BUCKET | AWS_REGION | AWS_ACCESS_KEY_ID | AWS_SECRET_ACCESS_KEY | CELERY_BROKER_URL
```

5. API Reference

Base URL: https://api.yourapp.com · Protected routes require: Authorization: Bearer <token>

5.1 Auth Service :8001

Method	Endpoint	Request / Params	Response
POST	/api/v1/auth/register	{ email, password, full_name, org_name }	{ user_id, message }
POST	/api/v1/auth/login	{ email, password }	{ access_token, refresh_token, expires_in }
POST	/api/v1/auth/refresh	{ refresh_token }	{ access_token }
POST	/api/v1/auth/logout [auth]	{ refresh_token }	{ message }
GET	/api/v1/auth/me [auth]	—	{ id, email, role, org_id, created_at }
PUT	/api/v1/auth/me [auth]	{ full_name?, avatar? }	{ user }
POST	/api/v1/auth/change-password [auth]	{ old_password, new_password }	{ message }

5.2 Mention Service :8002

Method	Endpoint	Request / Params	Response
POST	/api/v1/keywords [auth]	{ keyword, sources[], alert_threshold }	{ keyword_id, created_at }
GET	/api/v1/keywords [auth]	—	{ keywords[], total }
DELETE	/api/v1/keywords/{id} [auth]	—	{ message }
GET	/api/v1/mentions [auth]	?keyword&source;&sentiment;&from;_date&page;	{ mentions[], total, page }
GET	/api/v1/mentions/{id} [auth]	—	{ mention, sentiment, keywords[] }
POST	/api/v1/mentions/scrape [auth]	{ keyword_ids[] }	{ job_id, status }

5.3 NLP Service :8003

Method	Endpoint	Request / Params	Response
POST	api/v1/nlp/analyze [auth]	{ text, language? }	{ sentiment, confidence, keywords[], entities[] }
POST	api/v1/nlp/batch [auth]	{ texts[], callback_url? }	{ job_id, status }
GET	api/v1/nlp/jobs/{job_id} [auth]	—	{ status, progress, results[] }
GET	api/v1/nlp/summary/{id} [auth]	—	{ summary, key_points[] }

5.4 Analytics Service :8004

Method	Endpoint	Request / Params	Response
GET	/api/v1/analytics/overview [auth]	?org_id&from;_date&to;_date	{ total_mentions, positive_pct, negative_pct, avg_per_day }
GET	/api/v1/analytics/trends [auth]	?keyword&granularity;&from;_date	{ datapoints[{ time, count, sentiment }] }

GET	/api/v1/analytics/sentiment [auth]	?keyword&from;_date&to;_date	{ positive, negative, neutral, timeline[] }
GET	/api/v1/analytics/keywords/top [auth]	?limit&from;_date	{ keywords[{ word, count, sentiment }] }
GET	/api/v1/analytics/spikes [auth]	?from_date&to;_date	{ spikes[{ time, keyword, magnitude }] }
GET	/api/v1/analytics/competitors [auth]	?brands[]&metric;&from;_date	{ comparison[{ brand, metrics }] }
GET	/api/v1/analytics/sources [auth]	?from_date&to;_date	{ sources[{ name, count, sentiment }] }

5.5 Notification Service :8005

Method	Endpoint	Request / Params	Response
GET	/api/v1/alerts/rules [auth]	—	{ rules[] }
POST	/api/v1/alerts/rules [auth]	{ name, condition, channels[] }	{ rule_id }
PUT	/api/v1/alerts/rules/{id} [auth]	{ condition?, channels?, enabled? }	{ rule }
DELETE	/api/v1/alerts/rules/{id} [auth]	—	{ message }
GET	/api/v1/alerts/history [auth]	?from_date&to;_date&page;	{ alerts[], total }
WS	/ws/alerts?token={jwt} [auth]	—	{ type: alert|ping, payload }
WS	/ws/dashboard?token={jwt} [auth]	—	{ type: metrics_update, payload }

5.6 Report Service :8006

Method	Endpoint	Request / Params	Response
POST	/api/v1/reports [auth]	{ type: pdf|csv, filters }	{ report_id, status: queued }
GET	/api/v1/reports [auth]	?page&limit;	{ reports[], total }
GET	/api/v1/reports/{id} [auth]	—	{ status, download_url?, expires_at? }
DELETE	/api/v1/reports/{id} [auth]	—	{ message }
POST	/api/v1/reports/schedule [auth]	{ cron: daily|weekly, format, filters }	{ schedule_id }

6. Database Schema — PostgreSQL

Relationships: organizations 1→N users · organizations 1→N keywords · keywords 1→N mentions · mentions 1→1 sentiment_results · organizations 1→N alert_rules · alert_rules 1→N alerts · organizations 1→N reports

Table: users (12 columns)

Column	Data Type	Constraints	Notes
■ id	UUID	PRIMARY KEY DEFAULT gen_random_uuid()	
email	VARCHAR(255)	UNIQUE NOT NULL	Indexed
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash
full_name	VARCHAR(100)	NOT NULL	
role	ENUM(admin, analyst, viewer)	NOT NULL DEFAULT analyst	RBAC
■ org_id	UUID	FK → organizations.id NOT NULL	
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	
is_verified	BOOLEAN	NOT NULL DEFAULT FALSE	Email verified
avatar_url	TEXT	NULLABLE	
last_login_at	TIMESTAMPTZ	NULLABLE	
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	Auto-updated

Table: organizations (6 columns)

Column	Data Type	Constraints	Notes
■ id	UUID	PRIMARY KEY	
name	VARCHAR(200)	UNIQUE NOT NULL	
slug	VARCHAR(100)	UNIQUE NOT NULL	URL-safe name
plan	ENUM(free, pro, enterprise)	NOT NULL DEFAULT free	
max_keywords	INTEGER	NOT NULL DEFAULT 5	Plan keyword limit
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	

Table: keywords (8 columns)

Column	Data Type	Constraints	Notes
■ id	UUID	PRIMARY KEY	
■ org_id	UUID	FK → organizations.id NOT NULL	
keyword	VARCHAR(200)	NOT NULL	Tracked term
sources	TEXT[]	NOT NULL DEFAULT '{}'	['reddit','news',...]
alert_threshold	INTEGER	NOT NULL DEFAULT 10	Mentions/hr for alert
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	
■ created_by	UUID	FK → users.id	

created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	

Table: mentions (13 columns)

Column	Data Type	Constraints	Notes
■ id	UUID	PRIMARY KEY	
■ org_id	UUID	FK → organizations.id NOT NULL	
■ keyword_id	UUID	FK → keywords.id NOT NULL	
source	VARCHAR(50)	NOT NULL	reddit|news|blog|rss
source_url	TEXT	UNIQUE NOT NULL	Deduplication key
title	TEXT	NULLABLE	
content	TEXT	NOT NULL	Raw mention text
author	VARCHAR(200)	NULLABLE	
published_at	TIMESTAMPTZ	NOT NULL	
scraped_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	
upvotes	INTEGER	NOT NULL DEFAULT 0	
comment_count	INTEGER	NOT NULL DEFAULT 0	
language	VARCHAR(10)	NOT NULL DEFAULT en	ISO 639-1

Table: sentiment_results (12 columns)

Column	Data Type	Constraints	Notes
■ id	UUID	PRIMARY KEY	
■ mention_id	UUID	FK → mentions.id UNIQUE NOT NULL	1-to-1
sentiment	ENUM(positive, negative, neutral)	NOT NULL	
confidence	FLOAT	NOT NULL	0.0 to 1.0
positive_score	FLOAT	NOT NULL	
negative_score	FLOAT	NOT NULL	
neutral_score	FLOAT	NOT NULL	
keywords	TEXT[]	NOT NULL DEFAULT '{}'	Extracted terms
entities	JSONB	NOT NULL DEFAULT '[]'	[{text,label,score}]
summary	TEXT	NULLABLE	GPT-4o summary
model_version	VARCHAR(50)	NOT NULL	
analyzed_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	

Table: alert_rules (9 columns)

Column	Data Type	Constraints	Notes
■ id	UUID	PRIMARY KEY	
■ org_id	UUID	FK → organizations.id NOT NULL	
name	VARCHAR(200)	NOT NULL	

■ keyword_id	UUID	FK → keywords.id NULLABLE	
condition	JSONB	NOT NULL	{type, threshold, window_minutes}
channels	TEXT[]	NOT NULL	['email','websocket']
is_enabled	BOOLEAN	NOT NULL DEFAULT TRUE	
■ created_by	UUID	FK → users.id	
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	

Table: alerts (9 columns)

Column	Data Type	Constraints	Notes
■ id	UUID	PRIMARY KEY	
■ org_id	UUID	FK → organizations.id NOT NULL	
■ rule_id	UUID	FK → alert_rules.id NOT NULL	
keyword	VARCHAR(200)	NOT NULL	
trigger_reason	TEXT	NOT NULL	
mention_count	INTEGER	NOT NULL	
channel	ENUM(email, websocket, push)	NOT NULL	
is_read	BOOLEAN	NOT NULL DEFAULT FALSE	
triggered_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	

Table: reports (11 columns)

Column	Data Type	Constraints	Notes
■ id	UUID	PRIMARY KEY	
■ org_id	UUID	FK → organizations.id NOT NULL	
■ created_by	UUID	FK → users.id NOT NULL	
type	ENUM(pdf, csv)	NOT NULL	
status	ENUM(queued, processing, done, failed)	NOT NULL DEFAULT queued	
filters	JSONB	NOT NULL	{from_date, to_date, keywords[]}
s3_key	TEXT	NULLABLE	S3 object key
file_size_bytes	BIGINT	NULLABLE	
expires_at	TIMESTAMPTZ	NULLABLE	Pre-signed URL expiry
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	
completed_at	TIMESTAMPTZ	NULLABLE	

Recommended Indexes

```
CREATE INDEX idx_mentions_keyword_scraped ON mentions(keyword_id, scraped_at DESC); CREATE INDEX idx_mentions_org_scraped ON mentions(org_id, scraped_at DESC); CREATE INDEX idx_sentiment_mention ON sentiment_results(mention_id); CREATE INDEX idx_alerts_org_triggered ON alerts(org_id, triggered_at DESC); CREATE INDEX idx_mentions_source_url ON mentions(source_url); CREATE INDEX idx_mentions_published ON mentions(published_at DESC); CREATE INDEX idx_mentions_sentiment ON mentions(org_id) INCLUDE (sentiment_results.sentiment);
```

7. Kafka Event Architecture

Kafka is the backbone of all asynchronous communication. Producer services publish events and consumer services process them independently, ensuring loose coupling. Consumer groups guarantee ordered delivery within partitions. Failed messages go to a Dead Letter Topic (DLT).

7.1 Event Flow

Step	Producer Service	Kafka Topic	Consumer Service(s)
1	Mention Service	mention-created	NLP Service
2	NLP Service	sentiment-processed	Analytics Service · Notification Service
3	Analytics Service	analytics-updated	Notification Service
4	Analytics Service	alert-triggered	Notification Service
5	Report Service	report-generated	Notification Service

7.2 Topic Config & Payloads

Topic	Partitions	Replication	Retention	Producer	Consumers
mention-created	6	3	7 days	Mention Service	NLP Service

```
event_id: "uuid" mention_id: "uuid" org_id: "uuid" keyword_id: "uuid" source: "reddit" content: "Brand mentioned in discussion..." published_at: "2024-01-15T10:30:00Z" source_url: "https://reddit.com/r/tech/..."
```

Topic	Partitions	Replication	Retention	Producer	Consumers
sentiment-processed	6	3	7 days	NLP Service	Analytics Service · Notification Service

```
event_id: "uuid" mention_id: "uuid" org_id: "uuid" sentiment: "positive" confidence: 0.94 positive_score: 0.94 negative_score: 0.04 neutral_score: 0.02 keywords: ["brand", "excellent", "product"] entities: [{"text": "BrandX", "label": "ORG", "score": 0.99}]
```

Topic	Partitions	Replication	Retention	Producer	Consumers
analytics-updated	3	3	3 days	Analytics Service	Notification Service

```
event_id: "uuid" org_id: "uuid" keyword_id: "uuid" window: "1h" metrics: { mention_count: 47, positive_pct: 0.72, spike_detected: true, spike_magnitude: 3.2, z_score: 2.8 }
```

Topic	Partitions	Replication	Retention	Producer	Consumers
alert-triggered	3	3	3 days	Analytics Service	Notification Service

```
event_id: "uuid" org_id: "uuid" rule_id: "uuid" keyword: "BrandX" trigger_type: "spike" description: "Mention spike 3.2x above normal (z-score: 2.8)" severity: "high" triggered_at: "2024-01-15T11:00:00Z"
```

Topic	Partitions	Replication	Retention	Producer	Consumers
report-generated	2	3	1 day	Report Service	Notification Service

```
event_id: "uuid" report_id: "uuid" org_id: "uuid" created_by: "uuid" type: "pdf" s3_key: "reports/org-id/report-2024-01-15.pdf" file_size_bytes: 245678 generated_at: "2024-01-15T11:05:00Z"
```

8. Redis Design Patterns

Key Pattern	Redis Type	TTL	Purpose
`session:{user_id}`	HASH	15 min	JWT session data and user context
`blacklist:{jti}`	STRING	15 min	Invalidated JWT tokens (logout)
`rate:{user_id}:{60s-window}`	INCR+EXPIRE	60 sec	Sliding-window rate limiting counter
`analytics:{org_id}:{metric}`	HASH	5 min	Cached analytics query results
`ws:channel:{org_id}`	PUB/SUB	N/A	Real-time dashboard updates (WS bridge)
`ws:alerts:{org_id}`	PUB/SUB	N/A	Real-time alert delivery (WS bridge)
`dedup:{sha256_url}`	STRING	24 hrs	Mention URL deduplication (SET key)
`scrape:lock:{keyword_id}`	SETNX	10 min	Distributed scrape job lock
`trending:{org_id}`	ZSET	1 hr	Trending keywords sorted by score
`mention:count:{org_id}:{hour}`	INCR	48 hrs	Hourly mention counter
`nlp:summary:{mention_id}`	STRING	1 hr	GPT-4o summary cache
`report:progress:{report_id}`	HASH	24 hrs	Report generation progress %

9. gRPC Internal Communication

gRPC + Protocol Buffers provides type-safe, high-speed synchronous communication between services needing immediate responses. gRPC servers run on internal-only ports not exposed outside the Docker network. All stubs are auto-generated from .proto files.

Client Service	Server Service	RPC Methods	Use Case
Report Service	Analytics Service	GetAnalyticsSummary, GetTrends	Embed analytics data in PDF
Notification Service	Analytics Service	GetCurrentMetrics	Enrich alert payloads
Report Service	NLP Service	GetSentimentBatch	Bulk sentiment for reports
Analytics Service	NLP Service	GetEntitiesForMention	Entity extraction for keywords

```
syntax = "proto3"; package analytics.v1; service AnalyticsService { rpc GetAnalyticsSummary (SummaryRequest) returns (SummaryResponse); rpc GetTrends (TrendsRequest) returns (TrendsResponse); rpc GetCurrentMetrics (MetricsRequest) returns (MetricsResponse); } message SummaryRequest { string org_id = 1; string from_date = 2; string to_date = 3; } message SummaryResponse { int64 total_mentions = 1; float positive_pct = 2; float negative_pct = 3; float avg_per_day = 4; bool spike_detected = 5; }
```

10. Frontend Architecture

State Management

- Zustand store slices: auth, dashboard, alerts, reports, settings
- React Query (TanStack) for server state: cache, staleTime, refetchInterval
- Local state with useState / useReducer for UI-only state

Data Fetching

- Axios instance with baseURL; request interceptor attaches Authorization header
- Response interceptor: auto-refresh on 401 via /auth/refresh; retry original request
- React Query with staleTime=60s; analytics pages use refetchInterval=30s

Real-Time WebSocket

- useWebSocket custom hook: manages connection lifecycle with auto-reconnect
- Exponential backoff: 1s → 2s → 4s → 8s → 30s max; adds ±20% jitter
- Ping/pong keepalive every 30s; dispatches to Zustand store on every message

Charts & Visualisation

- Recharts: LineChart (trends), AreaChart (volume), BarChart (competitors), PieChart (sentiment)
- react-wordcloud for keyword cloud; animated on data change
- Custom KPI cards with CSS transition animations on value change

Pages & Routing

- React Router v6 createBrowserRouter; nested routes with layout component
- PrivateRoute HOC checks Zustand auth state; redirects to /login if unauthenticated
- Lazy-loaded pages with React.lazy + Suspense; skeleton loading states

Forms & UX

- React Hook Form + Zod schema validation; errors shown inline
- Infinite scroll with Intersection Observer API for mention feed
- Toast notifications via react-hot-toast; dark mode default via Tailwind dark: prefix

11. Development Roadmap — 15 Phases

Estimated: 6–8 weeks for complete production build. Phases 1–5: infrastructure skeleton · Phases 6–11: core features · Phases 12–15: hardening, monitoring, cloud deployment.

Phase 1: Foundation & Setup — 2–3 days

Monorepo init	GitHub repo, folder structure: frontend/, api-gateway/, auth-service/, mention-service/, nlp-service/, analytics-service/, notification-service/, report-service/, kafka/, monitoring/, nginx/, docker-compose.yml, docs/
Docker Compose	postgres:15, redis:7, kafka:3.6, zookeeper. Named volumes, custom bridge network, healthcheck probes per container.
Shared utilities	common/ package: loguru JSON logger, base exception hierarchy, Pydantic base models, correlation ID context var, UUID utils, config base class.
Environment config	.env.example per service, Pydantic Settings with env_file, startup validation raises clear errors on missing required vars.

Phase 2: API Gateway — 2 days

FastAPI scaffold	main.py with lifespan context, routers/, middleware/ (logging, CORS, security headers), dependencies/ for auth injection.
JWT middleware	Dependency decodes + validates JWT every protected route. Extracts user_id, role, org_id → Request.state.
Rate limiting	Redis sliding-window INCR+EXPIRE per (user_id, 60s window). 429 + Retry-After header on limit exceeded.
Request proxying	httpx.AsyncClient with timeout + retry (3×). Propagate X-Correlation-ID. Forward all headers except Authorization.

Phase 3: Auth Service — 3 days

DB + Alembic	Async SQLAlchemy + asyncpg driver. Alembic migration for users + organizations tables with all constraints.
Registration	Validate email uniqueness, bcrypt hash (cost=12), create org + user in single transaction, send verification email.
Login + JWT	bcrypt.checkpw (constant-time). Access token HS256 15 min. Refresh token UUID4 in Redis HASH 7-day TTL.
Refresh / Logout	POST /refresh: validates + rotates refresh token. POST /logout: deletes Redis key immediately.
RBAC	Role ENUM in DB. require_role(*roles) FastAPI dependency. Role encoded as JWT claim. Admin/Analyst/Viewer tiers.

Phase 4: Database Schema — 1 day

All 8 migrations	Alembic files: users, organizations, keywords, mentions, sentiment_results, alert_rules, alerts, reports.
Indexes	mentions(keyword_id, scraped_at DESC), mentions(org_id), sentiment_results(mention_id), alerts(org_id, triggered_at DESC), mentions(source_url).
Seed data	Faker scripts: 2 orgs, 5 users each, 10 keywords per org, 500 mentions + sentiment for dashboard testing.

Phase 5: Kafka Setup — 1 day

Local broker	Extend docker-compose: kafka + zookeeper. KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092. Verify with kafka-topics.sh.
Create topics	mention-created (6 partitions), sentiment-processed (6), analytics-updated (3), alert-triggered (3), report-generated (2).

Producer utility	JSON serializer, event_id + timestamp in every message, exponential retry on transient errors, graceful close.
Consumer base class	Abstract BaseConsumer: configurable group_id, manual offset commit, DLT publish on repeated failure.

Phase 6: Mention Collection Service — 4 days

Keyword CRUD	POST/GET/DELETE /keywords with auth + org scoping. Enforce plan max_keywords limit before insert.
Reddit scraper	PRAW OAuth2 client credentials. Search subreddits by keyword. Handle pagination and API rate limits.
NewsAPI scraper	/everything endpoint, sort by publishedAt, paginate results. Parse title, description, content, source.
RSS/Blog scraper	feedparser for 50+ RSS feeds. Playwright headless Chromium for JS-rendered pages. readability-lxml for content.
Deduplication	SHA256(source_url) as Redis SET key 24h TTL. UNIQUE DB constraint as second guarantee.
Celery scheduler	Celery Beat database scheduler. Task per active keyword with configurable interval. Flower at :5555.
Kafka publish	After DB insert, produce mention-created with org_id as partition key. Include correlation_id in headers.

Phase 7: NLP Service — 4 days

Kafka consumer	Consumer on mention-created, group=nlp-service. asyncio TaskGroup processes 10 concurrent. Manual commit after DB write.
Sentiment model	cardiffnlp/twitter-roberta-base-sentiment-latest loaded at startup. Batch inference (32 items). GPU via device='cuda'.
spaCy NER	en_core_web_sm pipeline. Extract ORG, PERSON, GPE, PRODUCT entities. Deduplicate by text+label. Store as JSONB.
Keyword extraction	TF-IDF vectorizer + KeyBERT semantic keywords. Merge, deduplicate, return top 5 ranked by relevance.
GPT summarisation	Content > 500 chars → GPT-4o-mini 2-sentence summary. Redis cache 1h TTL by mention_id. Skip if cached.
Publish	Write to sentiment_results. Publish sentiment-processed with mention_id as partition key.

Phase 8: Analytics Service — 3 days

Kafka consumer	Consumer on sentiment-processed. Update in-memory rolling counters per (org_id, keyword_id, window). Flush to DB every 60s.
Aggregation	Pandas groupby: org_id + time bucket (1h/24h/7d). Compute positive/negative/neutral counts per window.
Spike detection	Per keyword: 7-day rolling array of hourly counts. Mean + 2×std threshold. Publish alert-triggered on spike.
REST APIs	7 endpoints with Redis cache-aside (5-min TTL). Cache key = hash(endpoint + org_id + query_params).
Competitor scoring	Normalise mention count + sentiment to [0,1]. Composite = 0.6×mentions + 0.4×sentiment. Ranked response.

Phase 9: Real-Time WebSockets — 2 days

WS endpoints	FastAPI /ws/dashboard + /ws/alerts. JWT auth on HTTP upgrade handshake via query param token=.
Redis bridge	Kafka event received → redis.publish(f'ws:{org_id}', payload). WS handler subscribes + forwards to clients.
Scale support	Any service instance serves any user via shared Redis channel. No sticky sessions needed.
Frontend hook	useWebSocket: auto-reconnect with exponential backoff + jitter. Ping/pong keepalive 30s. Zustand dispatch on message.

Phase 10: Notification Service — 2 days

Alert rules CRUD	Full CRUD alert_rules. Condition JSON: {type, threshold, window_minutes, sentiment_filter}.

Evaluation engine	On analytics-updated: evaluate all enabled org rules. Debounce: max 1 alert per rule per 10 min.
SendGrid email	HTML Jinja2 template: spike summary, mention count, sentiment bars. Async send via SendGrid SDK.
Alert history	Persist all triggered alerts. GET /alerts/history with keyset pagination on triggered_at.

Phase 11: Report Service — 2 days

Async report API	POST /reports → 202 + Celery task. GET /reports/{id} polls status from DB.
PDF generation	Jinja2 HTML → WeasyPrint PDF. Charts as base64 PNG (Matplotlib). Cover, KPIs, trend chart, mentions table.
CSV export	Pandas JOIN mentions + sentiment_results. StreamingResponse for large files. All fields included.
S3 + URLs	boto3 multipart upload. 24h pre-signed URL. s3_key + expires_at stored in DB.

Phase 12: gRPC Communication — 2 days

Proto definitions	analytics.proto + nlp.proto with all service + message definitions. Package analytics.v1 / nlp.v1.
gRPC servers	Analytics :50051, NLP :50052. asyncio gRPC server alongside FastAPI in same process.
gRPC clients	Report + Notification services use grpc.aio async stubs to call Analytics + NLP servers.
Error handling	gRPC status codes → HTTP: NOT_FOUND→404, UNAVAILABLE→503. tenacity retry on UNAVAILABLE.

Phase 13: Frontend Development — 5 days

Scaffold	Vite + React 18 + TypeScript. Tailwind CSS custom dark theme. React Router v6. Zustand slices + React Query.
Auth pages	Login + Register with React Hook Form + Zod. Axios interceptor auto-refreshes JWT on 401.
Dashboard	KPI cards + AreaChart (Recharts) + last 10 alerts. Live WS updates from /ws/dashboard.
Mention feed	Virtualised infinite scroll. Filters: source, sentiment, keyword, date. Detail modal with NLP results.
Analytics page	Date picker. Tabs: Trends, Sentiment, Keywords (WordCloud), Competitors, Sources.
Alerts + Reports	Alert rules table + condition builder modal. Report form + status polling + S3 download.

Phase 14: Monitoring & Logging — 2 days

Prometheus	prometheus_fastapi_instrumentator per service. Custom metrics: kafka lag, NLP latency, scrape duration.
Grafana	Docker provisioning for datasources + dashboards. FastAPI, Kafka, Redis, DB, business metrics boards.
Logging	loguru JSON sink → stdout. Promtail → Loki. correlation_id in every log line. Sentry for errors.
Health checks	GET /health: DB ping, Redis ping, Kafka producer test. Returns {status, checks, version, uptime}.

Phase 15: AWS Deployment + CI/CD — 3–4 days

Dockerfiles	Multi-stage builds: builder installs deps, final stage copies only needed files. Non-root USER. < 200MB.
ECR	GitHub Actions: docker build + tag with git SHA + push to ECR on merge to main. Scan with ECR.
AWS infra	RDS PostgreSQL 15 Multi-AZ · ElastiCache Redis cluster · MSK 3-broker · S3 versioned · ALB HTTPS.
ECS	Task definitions per service (CPU/memory limits). ECS Service desired=2, rolling update. ALB target group.
CI/CD	GitHub Actions: pytest + coverage → ruff lint → build → ECR push → ECS task update → deploy.

Secrets	AWS Secrets Manager per service. ECS task IAM role reads specific ARNs. Zero secrets in code/env.

12. AWS Deployment Architecture

AWS Service	Configuration	Purpose
EC2 / ECS	t3.medium ×2 per service, auto-scaling group	Run all 7 microservice Docker containers
RDS PostgreSQL	db.t3.medium, Multi-AZ, PostgreSQL 15, 100GB	Primary persistent datastore
ElastiCache	cache.r6g.large, Redis 7, cluster mode ON	Caching, pub/sub, rate limiting, sessions
MSK Kafka	kafka.m5.large ×3 brokers, replication-factor 3	Managed Kafka for event streaming
S3	Standard class, versioning enabled, lifecycle rules	PDF/CSV reports, logs, static assets
ALB	HTTPS :443, HTTP→HTTPS redirect, WAF attached	load balancing, SSL termination
Route53	A-record alias → ALB, failover health checks	DNS management
CloudWatch	Log groups per service, metric alarms on error rate	Centralised logs, alerting
ECR	Private repo per service, image scanning ON	Docker image registry and vulnerability scanning
Secrets Manager	One secret per service, IAM task role access	Secure credential management
VPC	Private subnets for services, public subnet for ALB	Network isolation and security

13. Monitoring & Observability

Prometheus Metrics to Collect

- http_requests_total{service, method, endpoint, status_code} — request rate and error rate per service
- http_request_duration_seconds{service, endpoint} — P50/P95/P99 latency histograms
- kafka_messages_consumed_total{service, topic, group} — throughput tracking per consumer
- kafka_consumer_lag{topic, partition} — backpressure detection; alert if lag > 1000
- nlp_inference_duration_seconds{model, batch_size} — AI model latency monitoring
- scrape_job_duration_seconds{source, keyword} — scraper performance tracking
- db_connection_pool_used{service} — connection pool exhaustion detection
- redis_cache_hit_ratio{pattern} — effectiveness of caching layer

Grafana Dashboards

- System Overview: all services request rate, error rate, latency on a single pane
- Kafka Dashboard: messages/sec per topic, consumer lag per partition, broker health
- Database Dashboard: query duration, connection count, index hit ratio, table bloat
- Redis Dashboard: memory usage, hit/miss ratio, connected clients, ops/sec
- NLP Pipeline: inference latency, batch sizes, model throughput, Kafka queue depth
- Business Metrics: mentions per hour, sentiment trend, alerts fired, reports generated per org

Logging Strategy

- All services emit structured JSON logs via loguru to stdout; collected by Docker log driver
- Log levels: DEBUG (dev), INFO (prod default), WARNING/ERROR (always), CRITICAL (PagerDuty)
- Every log line includes: timestamp, level, service_name, correlation_id, user_id (if available)
- Promtail sidecar ships container logs to Grafana Loki; 30-day retention policy
- Error-level logs automatically create Sentry issues with stack trace and request context
- Audit logs (login, logout, password change, keyword create/delete) stored in separate DB table

This document provides a complete technical blueprint for the AI-Powered Social Listening & Reputation Monitoring Platform. Following these specifications produces a production-grade system that demonstrates distributed systems, AI integration, real-time engineering, and cloud deployment — an ideal flagship project for senior backend or full-stack engineering roles.